AWS IAM Security Review Framework

How a Principal Cloud Security Engineer Reviews IAM

Covers: Review methodology · 15 dangerous misconfigurations · 8 full exploit chains
Domains: Identity · Access · Trust · Permissions · Detection · Org-level controls

Prepared for: Product Security Engineer → Cloud Security role interview

Contents

SECTION 01

How a Principal CSE Reviews IAM

An effective IAM security review is not a checklist exercise — it is a threat-modelling exercise. The question is not 'does this policy comply with policy X?' but 'what can an attacker with these permissions actually do, and what is the worst-case blast radius if this identity is compromised?'

The review methodology — three passes

Pass	Focus	Primary tools	Output
1 — Posture scan	Account-level baseline: root, MFA, access keys, SCPs	IAM credential report, Trusted Advisor, Security Hub CIS	Pass/fail on 10 baseline controls
2 — Permission depth	Every role and user: what can they actually do? Escalation paths?	IAM Access Analyzer, IAM policy simulator, Cloudsplaining	Ranked list of over-privileged identities
3 — Trust analysis	Who can assume what? External access? OIDC conditions? Cross-account?	Access Analyzer findings, manual trust policy review	External access map + trust graph

What to look at first — priority order

As a principal CSE you have finite time. This is the triage order that maximises impact:

P0 — Root account	Active root access keys or root used in last 30 days. Stop everything and fix this first.
P0 — CloudTrail disabled	If logging is off, you are blind. Re-enable and check for the gap window.
P1 — Wildcard policies	Action:* Resource:* on any identity. Every IAM escalation primitive is already granted.
P1 — PassRole with Resource:*	The single most common escalation path in AWS breach reports.
P2 — Long-lived access keys	Keys older than 90 days. Check last-used date — unused keys are dormant backdoors.
P2 — Cross-account trust without ExternalId	Especially common with third-party SaaS integrations.
P3 — Inline policies	Hidden permissions that survive standard audits.
P3 — OIDC roles without branch conditions	Common in GitHub Actions setups, often overlooked.

Tool stack for IAM review

Tool	Purpose	Key output
IAM Credential Report	Snapshot of all users: MFA, access key age, last login	CSV — pivot for P0/P1 findings
IAM Access Analyzer	Finds resources with external access, generates least-privilege policies	Findings by resource type
Cloudsplaining (open source)	Analyses all IAM policies for privilege escalation paths	HTML report with escalation paths
Prowler	900+ AWS security checks including IAM, CIS benchmark	JSON/CSV severity-ranked findings
AWS Config Rules	Continuous compliance: MFA, key rotation, inline policies, public access	Config dashboard + SNS alerts
CloudTrail + Athena	Historical API call analysis — who called what, when, from where	SQL queries over 90-day log history
IAM Policy Simulator	Test what a specific identity can actually do before deploying changes	Allow/deny result per action

SECTION 02

IAM Identity & Account Baseline

The account baseline is the floor of your IAM security posture. No amount of fine-grained policy work matters if root has no MFA, or if developers have long-lived access keys rotating every never.

15 dangerous IAM misconfigurations

Ranked by real-world exploitation frequency and blast radius:

#	Severity	Misconfiguration	Category	Impact summary
01	CRITICAL	Action:* Resource:* wildcard policy	Policy	Equivalent to AdministratorAccess on any identity that holds it. Full account takeover on credential leak.
02	CRITICAL	Root account active access keys	Account	Root ignores all SCPs and permission boundaries. Cannot be restricted — only remediated.
03	CRITICAL	No MFA on root or privileged users	Account	Password phish → immediate full access. Zero detection window before attacker acts.
04	CRITICAL	Long-lived access keys for humans	Credentials	Keys leaked to GitHub exploited by automated scanners in under 2 minutes on average.
05	CRITICAL	iam:PassRole with Resource:*	Policy	Attach AdminRole to EC2/Lambda. IMDS credential theft gives admin STS credentials.
06	CRITICAL	iam:CreatePolicyVersion unrestricted	Policy	Single API call self-escalation to admin — no new resources, zero footprint.
07	HIGH	iam:AttachUserPolicy on non-admin role	Policy	One API call attaches AdministratorAccess to own user — instant escalation.
08	HIGH	Cross-account trust with no ExternalId	Trust	Confused deputy: any principal in trusted account can assume the high-priv role.
09	HIGH	GitHub OIDC role without branch condition	Trust	Any PR from any fork assumes the CI role — malicious PR deploys to production.
10	HIGH	Lambda secrets in environment variables	Credentials	lambda:GetFunctionConfiguration exposes all secrets in plaintext to broad audience.
11	HIGH	iam:* on developer or service role	Policy	Every IAM escalation primitive in one grant — functionally equivalent to admin.
12	HIGH	Resource-based policy with Principal:*	Policy	Any AWS account globally can access the resource — KMS key, S3 bucket, Lambda.
13	HIGH	Inline policies on IAM identities	Policy	Hidden from standard audits, deleted silently with the user — backdoor favourite.
14	MEDIUM	No permission boundaries on developer roles	Policy	Escalation primitive gained later is uncapped — no ceiling on blast radius.
15	MEDIUM	No SCP guardrails on member accounts	Account	Compromised account can disable CloudTrail, GuardDuty, and erase its own trail.

CRITICAL The top 6 critical misconfigurations account for >80% of AWS IAM-related breach incidents reported publicly. Fix these first before any fine-grained permission work.

SECTION 03

Policy & Permission Misconfigurations

Permission misconfigurations are the primary mechanism for privilege escalation in AWS. Understanding the escalation primitives and how they chain together is the core skill that separates a cloud security engineer from a generalist.

The escalation primitive map

Each of these permissions, when granted without restriction, enables a specific escalation path. They are the 'keys to the kingdom' — treat each one as a high-value finding on any access review:

Permission	Required pair	Escalation mechanism	Risk
iam:PassRole	ec2:RunInstances or lambda:CreateFunction	Attach high-priv role to controlled resource, steal via IMDS	CRITICAL
iam:CreatePolicyVersion	(self-contained)	Rewrite any managed policy to grant Action:* instantly	CRITICAL
iam:SetDefaultPolicyVersion	(self-contained if prior permissive version exists)	Switch default to previously created permissive version	HIGH
iam:AttachUserPolicy	(self-contained)	Attach AdministratorAccess to own user in one call	CRITICAL
iam:AttachRolePolicy	(self-contained)	Attach admin policy to any role — including ones attacker can assume	HIGH
iam:UpdateAssumeRolePolicy	sts:AssumeRole	Add own ARN to trust policy of high-priv role, then assume it	HIGH
iam:CreateAccessKey	iam:ListUsers	Create long-lived backdoor key on any user including dormant admins	HIGH
iam:CreateLoginProfile	(existing passwordless admin user)	Add console password to admin service account	MEDIUM
glue:CreateDevEndpoint	iam:PassRole	JupyterLab notebook running with passed role — often missed	MEDIUM
lambda:UpdateFunctionCode	(existing Lambda with high-priv role)	Overwrite function code to exfil role credentials on next invocation	HIGH

The policy evaluation trap: what most engineers miss

The six-step IAM policy evaluation order is not just theoretical — it has direct implications for how you scope fixes. Getting this wrong in a review means you believe something is blocked when it isn't:

Step 1	**Explicit Deny**	ANY explicit deny — anywhere in any policy — wins immediately. No exceptions.
Step 2	**SCPs**	Organization-level ceiling. Caps what is possible in the account. Cannot grant — only restrict.
Step 3	**Resource-based policy**	Attached to the resource itself. Can grant cross-account access independently.
Step 4	**Identity-based policy**	Attached to the user/role. Most common grant mechanism.
Step 5	**Permission boundary**	Caps identity-based policy grants. Does NOT grant permissions itself.
Step 6	**Session policy**	Further restricts role session. Passed in AssumeRole call.

NOTE The most common interview trap: 'Can a permission boundary grant permissions?' NO. It only caps. The actual allow must come from an identity-based policy. Same for SCPs.

SECTION 04

Trust & Federation Weaknesses

Trust policies define who can impersonate a role. A misconfigured trust policy is often more dangerous than a misconfigured permission policy — it can give an external attacker a direct door into your account without needing to compromise any of your own identities.

Trust policy vulnerability patterns

Pattern	Risk level	Attack scenario	Fix
Principal: *	CRITICAL	Any authenticated AWS identity globally can assume the role	Replace with explicit ARN + aws:PrincipalOrgID condition
Cross-account, no ExternalId	HIGH	Confused deputy — any principal in trusted account assumes the role	Add sts:ExternalId condition with secret agreed out-of-band
OIDC, no sub/ref condition	CRITICAL	Any PR from any fork of any repo assumes the role	StringLike on token:sub scoped to main branch + specific repo
OIDC, wrong claim checked	HIGH	Attacker crafts a repo name matching the condition	Check token:sub (repo:owner/name:ref:...) not just token:repository
Service: * in Principal	MEDIUM	Broader than needed — any AWS service can assume role	Scope to specific service: ec2.amazonaws.com, lambda.amazonaws.com
Stale trust — old account	MEDIUM	Account in Principal no longer owned by your org — 3rd party acquired it	Quarterly audit of all cross-account trust ARNs
No MFA condition on sensitive roles	HIGH	Password-only compromise of a federated user escalates to high-priv role	Add aws:MultiFactorAuthPresent condition on admin role assumptions

The External ID mechanism — why it exists

The External ID solves the confused deputy problem. Without it, if a third party is granted access to your account and they are compromised, an attacker can use that third party as a proxy to access your resources. The External ID is a secret known only to you and the trusted party — it cannot be guessed or brute-forced because AWS does not increment it on failure.

```
# Correct cross-account trust policy:
"Condition": {
  "StringEquals": {
    "sts:ExternalId": "unique-secret-agreed-out-of-band"
  }
}
# Wrong — ExternalId in the trust policy but disclosed publicly
# (e.g. in documentation or error messages) defeats the protection
```

OIDC condition — what a correct GitHub Actions trust looks like

```
# CORRECT — restricts to main branch of specific repo:
"Condition": {
  "StringLike": {
    "token.actions.githubusercontent.com:sub":
      "repo:myorg/myrepo:ref:refs/heads/main"
  }
}

# WRONG — checks repository name only (any branch, any fork):
"StringEquals": {
  "token.actions.githubusercontent.com:repository": "myorg/myrepo"
}
# Attacker creates repo named myorg/myrepo in their own org namespace
# — this condition is bypassable
```

EXPLOIT CHAIN 05

iam:PassRole → Full Admin

CRITICAL

Low-priv user delegates AdminRole to EC2, steals credentials via IMDS

Kill chain

1

Low-priv key obtained

Initial access

Leaked GitHub key, CI secret, or phished credentials. Identity has iam:PassRole + ec2:RunInstances. Attacker confirms via aws sts get-caller-identity.

2

Enumerate assumable roles

Discovery

aws iam list-roles lists all roles. Attacker identifies AdminRole or PowerUserAccess. Resource:* on PassRole means no restriction on which role can be passed.

3

Launch EC2 with AdminRole

Privilege escalation

ec2:RunInstances + iam:PassRole attaches AdminRole to new instance. Attacker controls the instance via SSH or SSM Session Manager.

4

Steal credentials via IMDS

Credential access

curl http://169.254.169.254/latest/meta-data/iam/security-credentials/AdminRole returns AccessKeyId, SecretAccessKey, Token, Expiration in JSON.

5

Account takeover

Impact

STS credentials used from external IP. Create new IAM user, disable CloudTrail, exfiltrate S3 data, pivot cross-account via existing trust relationships.

BLAST RADIUS Full account compromise. All data, all services, all accounts with trust relationships — reachable from a single low-priv key with PassRole + RunInstances.

Detection signals

Service	Event / Finding	What it means
CloudTrail	`RunInstances + PassRole same principal within 5 min`	High-signal escalation indicator
GuardDuty	`InstanceCredentialExfiltration.OutsideAWS`	Credentials used from non-AWS IP

Service	Event / Finding	What it means
CloudTrail	`CreateUser or AttachUserPolicy by EC2 role`	Unusual principal type for IAM writes

Controls that break the chain

Control	Implementation	Breaks step
Scope PassRole to ARN	Resource: arn:aws:iam::ACCT:role/AutomationRole only — not Resource:*	**Breaks step 3**
Permission boundary	Boundary on developer role caps max permissions of any passed role	**Breaks step 3**
IMDSv2 + hop-limit=1	Enforce org-wide via SCP. Blocks SSRF-based IMDS theft	**Breaks step 4**
EventBridge alert	RunInstances + PassRole from same principal within 5-min window	**Detects step 3**

PRODSEC ANGLE

PassRole is the cloud equivalent of setuid — a user delegates elevated authority to a process they control. Threat model any feature that provisions compute: 'can the requester attach a role more powerful than themselves?'

EXPLOIT CHAIN 06

iam:CreatePolicyVersion Self-Escalation

CRITICAL

Attacker rewrites managed policy to grant Action:* — zero new resources, zero footprint

Kill chain

1

Any role with CreatePolicyVersion

Initial access

Role has iam:CreatePolicyVersion (often granted for 'IAM management' work). Target: any managed policy attached to the attacker's identity.

2

Create permissive policy version

Privilege escalation

aws iam create-policy-version --policy-arn TARGET_POLICY --policy-document '{"Statement":[{"Effect":"Allow","Action":"*","Resource":"*"}]}' --set-as-default

3

Instant admin access

Impact

Policy version switch is immediate. Attacker's identity now has AdministratorAccess. No new EC2, no new roles, no IMDS — just IAM API calls.

BLAST RADIUS	The most surgical escalation path in AWS. Single API call, no new resources, immediate admin. Hardest to detect because CreatePolicyVersion is a legitimate operation.

Detection signals

Service	Event / Finding	What it means
CloudTrail	`CreatePolicyVersion with Action:* or Resource:* in new version`	Monitor policy document content
CloudTrail	`SetDefaultPolicyVersion immediately after CreatePolicyVersion`	Two-event escalation sequence
Config	`iam-no-inline-policy-check + custom rule for permissive versions`	Continuous compliance monitoring

Controls that break the chain

Control	Implementation	Breaks step
Deny CreatePolicyVersion	Except dedicated IAM governance role with MFA required	**Breaks step 2**

Control	Implementation	Breaks step
SCP in member accounts	Deny iam:CreatePolicyVersion in all non-management accounts	**Breaks step 2**
Config custom rule	Alert when any policy version gains Action:* or broad Resource:*	**Detects step 2**

PRODSEC ANGLE

This is the cloud equivalent of a developer editing their own sudoers entry. The threat model question: 'who can modify the policies attached to their own identity?' If the answer includes anyone other than a named IAM governance role, you have this finding.

EXPLOIT CHAIN 07

SSRF → IMDS → Lateral Movement

CRITICAL

Web application SSRF hits EC2 metadata service, stealing IAM role credentials

Kill chain

1

SSRF in web application

Initial access

URL fetch, PDF generator, image proxy, or webhook handler fetches attacker-controlled URL. Redirect points to 169.254.169.254 (IMDSv1) or crafted payload.

2

Enumerate instance role name

Discovery

GET http://169.254.169.254/latest/meta-data/iam/security-credentials/ returns the role name attached to the EC2 instance (e.g. web-server-role).

3

Steal role credentials

Credential access

GET .../security-credentials/web-server-role returns JSON with AccessKeyId, SecretAccessKey, Token valid for up to 12 hours.

4

Use credentials externally

Lateral movement

Credentials used from attacker's IP. If role has iam:PassRole or S3:* or Secrets Manager access — pivot across the account.

5

Establish persistence

Persistence

Create IAM user with keys, exfiltrate Secrets Manager values, copy S3 data. Credentials valid until expiry (~6hrs) even after SSRF is patched.

BLAST RADIUS Single SSRF vulnerability chains to full account compromise if EC2 role is over-permissioned and IMDSv1 is enabled. One of the most impactful web vulnerability classes in cloud environments.

Detection signals

Service	Event / Finding	What it means
GuardDuty	`UnauthorizedAccess:EC2/MetaDataDNSRebind`	DNS rebinding SSRF to IMDS

Service	Event / Finding	What it means
GuardDuty	`UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration.OutsideAWS`	Creds used from non-AWS IP
CloudTrail	`Any API call from EC2 role sourceIP = external non-AWS address`	IMDS creds used externally
VPC Flow Logs	`Outbound connection from EC2 to 169.254.169.254 from unexpected process`	SSRF network indicator

Controls that break the chain

Control	Implementation	Breaks step
IMDSv2 required	Enforce hop-limit=1 org-wide via SCP — PUT required, breaks most SSRF	**Breaks step 2**
EC2 role least privilege	Role should only have permissions the application actually needs	**Limits step 4**
SSRF input validation	Application layer: block RFC1918 + 169.254.x.x in URL allowlist	**Breaks step 1**
SCP: enforce IMDSv2	aws:EC2MetadataHttpTokens: required on all instance launches	**Breaks step 2**

PRODSEC ANGLE

As a prodsec engineer this is your home territory — SSRF is a classic web vulnerability. The cloud-specific insight is that the blast radius is magnified by the EC2 role. Threat model every feature that fetches external URLs: 'can this be redirected to 169.254.169.254?'

EXPLOIT CHAIN 08

CI/CD OIDC → Production Backdoor

CRITICAL

Malicious PR assumes overpermissive OIDC role, deploys attacker code to production

Kill chain

1

Malicious PR submitted

Initial access

Attacker submits PR to repo with GitHub Actions + AWS OIDC. Or compromises a contributor's GitHub account. Workflow runs on pull_request events.

2

OIDC role assumed — no branch condition

Privilege escalation

Workflow calls sts:AssumeRoleWithWebIdentity. Trust policy has no token:sub branch condition. Any PR from any fork assumes the deployment role.

3

Backdoor injected into build

Persistence

Malicious workflow step modifies application code before build. Inserts data exfil function, reverse shell trigger, or hidden admin endpoint.

4

Backdoor reaches production

Impact

Artifact signed, deployed to Lambda or ECS. Every production invocation now executes attacker code. User data exfiltration begins.

BLAST RADIUS Supply chain compromise of all users of the production service. Attacker controls code running in production — exfiltrates user data, credentials, uses service as C2 relay.

Detection signals

Service	Event / Finding	What it means
CloudTrail	`AssumeRoleWithWebIdentity from unexpected repo or branch in token claim`	Check federatedUser claim
CloudTrail	`UpdateFunctionCode or RegisterTaskDefinition from CI role on non-main branch`	Branch-sensitive deploy alert
CodeBuild	`Unexpected outbound connections or new packages in build logs`	Build anomaly detection

Controls that break the chain

Control	Implementation	Breaks step
Branch condition on OIDC	StringLike on token:sub: repo:myorg/myrepo:ref:refs/heads/main	**Breaks step 2**
Separate PR vs merge roles	PR role = read-only. Merge role = deploy. Never combine.	**Limits step 2**
AWS Signer	Sign artifacts. Runtime rejects unsigned images/packages.	**Breaks step 3**
Manual prod approval gate	Production deployment requires human approval in pipeline	**Breaks step 4**

PRODSEC ANGLE

Supply chain security is core prodsec. The cloud-specific layer: the IAM trust policy is the security control, not the application code. A missing branch condition in a trust policy is equivalent to a missing authentication check on an API endpoint.

EXPLOIT CHAIN 09

Cross-Account Role Chaining

CRITICAL

Dev account compromise pivots to production via unconstrained role assumption

Kill chain

1

Dev account credentials compromised

Initial access

Dev account has weaker controls: no MFA enforcement, long-lived keys, fewer SCPs. Phish, leaked key, or compromised CI pipeline token.

2

Enumerate cross-account roles

Discovery

aws iam list-roles + sts:AssumeRole attempts. Finds prod account role with Principal: arn:aws:iam::DEV-ACCT:root and no ExternalId requirement.

3

Assume prod account role

Lateral movement

sts:AssumeRole returns prod account STS credentials. Attacker now operates in production — having crossed the account boundary without touching the internet.

4

Access production data

Collection

Read prod RDS snapshots, S3 buckets, Secrets Manager secrets. Enumerate all ECS tasks, Lambda functions, EC2 instances. Full inventory of production assets.

5

Create prod backdoor

Persistence

CreateUser + AttachUserPolicy + CreateAccessKey in prod account. Backdoor survives dev credential rotation. Attacker retains prod access indefinitely.

BLAST RADIUS Dev account breach becomes prod breach. All production data, all production services, all production IAM — accessible from a single compromised dev credential.

Detection signals

Service	Event / Finding	What it means
CloudTrail	`AssumeRole in prod with sourceAccount = dev account`	Cross-account calls in target CloudTrail
IAM Access Analyzer	`External access finding on role without ExternalId`	Continuous cross-account analysis

Service	Event / Finding	What it means
CloudTrail	`CreateUser + AttachUserPolicy + CreateAccessKey sequence in prod`	3-event persistence pattern

Controls that break the chain

Control	Implementation	Breaks step
ExternalId required	Add sts:ExternalId condition — shared secret prevents confused deputy	**Breaks step 3**
SCP: deny cross-account assume	SCP on dev account denying AssumeRole to prod ARNs	**Breaks step 3**
Prod SCPs: deny CreateUser	Deny iam:CreateUser, iam:CreateAccessKey in prod OU	**Breaks step 5**
IAM Access Analyzer	Continuously identifies roles with unconstrained cross-account trust	**Detects step 2**

PRODSEC ANGLE

Account boundaries are not trust boundaries unless you enforce them. Every cross-account trust policy should be reviewed as a security-critical document with the same rigour as a firewall rule.

EXPLOIT CHAIN 10

Lambda Env Secrets → Multi-Service Pivot HIGH

lambda:GetFunctionConfiguration exposes all secrets → pivot into RDS, Stripe, JWT forge

Kill chain

1 **Role with GetFunctionConfiguration**

Initial access

Developer, ops, or CI role has lambda:GetFunctionConfiguration. Often granted for debugging. Attacker compromises this role via leaked key.

2 **Dump all Lambda env vars**

Collection

aws lambda list-functions | xargs aws lambda get-function-configuration returns Environment.Variables for every function — DB_PASSWORD, STRIPE_SECRET, JWT_SECRET in plaintext.

3 **Pivot to RDS**

Lateral movement

DB_PASSWORD used to connect directly to production RDS. Full read/write access to customer data — orders, PII, financial records.

4 **Forge JWT tokens**

Lateral movement

JWT_SECRET used to sign admin-level tokens. Attacker accesses any customer account, any admin panel, any internal API that trusts the token.

5 **Abuse payment API key**

Lateral movement

STRIPE_SECRET issues refunds, reads card data (last 4 + metadata), creates charges. Financial fraud + potential PCI violation.

BLAST RADIUS All secrets in all Lambda functions exposed via one IAM permission. RDS data, payment systems, and authentication all compromised from a single developer-tier role.

Detection signals

Service	Event / Finding	What it means
CloudTrail	`Bulk GetFunctionConfiguration across multiple functions in short window`	Automated harvesting pattern

Service	Event / Finding	What it means
CloudTrail	`GetFunctionConfiguration from unexpected source IP or principal`	Credential misuse signal
GuardDuty	`Anomalous API call spike from developer role`	Baseline deviation alert

Controls that break the chain

Control	Implementation	Breaks step
Secrets Manager	All secrets fetched at runtime from Secrets Manager — never in env vars	**Breaks step 2**
Least-priv: deny GetFunctionConfig	Remove lambda:GetFunctionConfiguration from non-ops roles	**Breaks step 2**
KMS-encrypt env vars	Transitional control — GetFunctionConfiguration returns ciphertext only	**Limits step 2**
Rotate on detection	Rotate all secrets immediately on any suspicious GetFunctionConfiguration	**Limits step 3-5**

PRODSEC ANGLE

This is the cloud equivalent of storing passwords in a config file on a shared filesystem. Secrets in Lambda env vars is an anti-pattern that fails safe design: the secret store (Secrets Manager) and the application runtime should be separate blast-radius boundaries.

EXPLOIT CHAIN 11

Public S3 + Embedded Credentials

HIGH

Public bucket contains config files with hardcoded credentials → full account pivot

Kill chain

1 **Discover public S3 bucket**

Reconnaissance

GrayhatWarfare, S3Scanner, or DNS enumeration finds company-name-backup.s3.amazonaws.com. Bucket lists objects without authentication.

2 **Download all objects**

Collection

aws s3 sync s3://company-name-backup ./local — terabytes of data including DB dumps, config files, .env files, deployment packages.

3 **Extract embedded credentials**

Credential access

grep -r 'AWS_SECRET\|DB_PASSWORD\|api_key' ./local finds credentials in .env files, terraform state, CI config, application config YAML.

4 **Use extracted credentials**

Lateral movement

AWS keys in terraform.tfstate give direct account access. Database passwords give production data access. API keys give third-party service access.

BLAST RADIUS Complete data exfiltration of bucket contents plus credential-based account compromise. If terraform state is present: full infrastructure visibility and likely account admin credentials.

Detection signals

Service	Event / Finding	What it means
CloudTrail	`Anonymous GetObject requests — no userIdentity on S3 data events`	Public access indicator
Macie	`Sensitive data discovery: AWS credentials, PII found in bucket`	Continuous content scanning
Security Hub	`S3.2 — S3 bucket should not be publicly accessible`	CIS benchmark finding

Controls that break the chain

Control	Implementation	Breaks step
S3 Block Public Access	Enable at account level — blocks all public access org-wide	**Breaks step 1**
Config: s3-bucket-public-read-prohibited	Auto-remediate any bucket that goes public	**Breaks step 1**
Secrets rotation	Rotate any credentials found immediately on discovery	**Limits step 4**
Macie enable	Continuous scanning catches PII/creds before attacker finds them	**Detects step 2**

PRODSEC ANGLE

Data exposure + credential exposure compound each other. Threat model S3 buckets by their worst-case contents: 'if this bucket were public tomorrow, what is the blast radius?' Build controls to that worst case, not to the current contents.

EXPLOIT CHAIN 12

Malicious ECR Image → Task Role Theft

HIGH

Poisoned container base image runs on ECS, steals task role credentials at startup

Kill chain

1

Base image poisoned

Supply chain

Attacker compromises Docker Hub base image (e.g. node:18-alpine) or npm package used in Dockerfile. Malicious layer added that runs at container startup.

2

Image enters ECR via CI

Execution

CI/CD pipeline pulls compromised base, builds on top, pushes to ECR. No image scanning in place. ECR stores poisoned image tagged as latest.

3

ECS deploys poisoned task

Execution

ECS pulls image, starts task. Malicious layer runs before application code — calls http://169.254.170.2/v2/credentials/TASK-UUID (ECS credential endpoint).

4

Steal task role credentials

Credential access

ECS credential endpoint returns task's IAM role AccessKeyId, SecretAccessKey, Token in plaintext JSON. Exfiltrated to attacker C2 over HTTPS.

5

Pivot from task role

Lateral movement

If task role has S3 read, Secrets Manager, iam:PassRole — attacker has those permissions externally. Worst case: PassRole present and chain continues.

BLAST RADIUS Attacker code executes in all production containers at startup. Task role credentials stolen on every new task launch — a persistent, renewable credential source.

Detection signals

Service	Event / Finding	What it means
ECR / Inspector	`Critical CVE or known malicious package hash on image push`	Pre-deployment gate
GuardDuty	`ECS Runtime Monitoring — unexpected outbound connection at task startup`	Runtime anomaly

Service	Event / Finding	What it means
CloudTrail	`GetObject or Secrets Manager access from ECS role at unusual hours`	Post-compromise access

Controls that break the chain

Control	Implementation	Breaks step
ECR enhanced scanning	Block deployment pipeline if Inspector finds critical CVEs on push	**Breaks step 2**
AWS Signer (image signing)	ECS task definitions reject unsigned images	**Breaks step 3**
Task role least privilege	Minimum permissions — if no AWS API calls needed, no role attached	**Limits step 4**
GuardDuty ECS runtime	Detects unexpected outbound connections from running containers	**Detects step 3**

PRODSEC ANGLE

Supply chain security and IAM least privilege are both your domain here. The combined threat model question: 'what is the most dangerous thing this container is allowed to do?' Threat model the task role as carefully as you'd threat model an admin user.

SECTION 13

Org-Level IAM Controls

Organisation-level controls are the force multiplier of IAM security. A well-designed SCP baseline applied to an entire OU eliminates entire classes of misconfiguration across hundreds of accounts simultaneously — before any developer has a chance to make the mistake.

Essential SCP baseline — apply to all member OUs

SCP control	What it prevents	Rationale
Deny cloudtrail:StopLogging	Attacker disabling audit trail post-compromise	No legitimate reason to stop logging in production
Deny guardduty:DeleteDetector	Attacker blinding detection layer	Defence-in-depth: detection cannot be disabled by the workload
Deny iam:CreateAccessKey for humans	Long-lived human credential creation	All human access should use SSO — no IAM user keys
Deny s3:PutBucketAcl (public)	Public bucket ACL misconfiguration	Redundant with Block Public Access but defence-in-depth
Deny actions outside approved regions	Data residency + lateral movement	Limits blast radius of account compromise to approved regions
Deny iam:CreateUser in prod	Backdoor IAM user creation post-compromise	All identities provisioned via IaC/SSO — no manual user creation
Deny disabling Security Hub	Attacker blinding compliance dashboard	Security Hub findings are critical for incident response
Deny root credentials use	Root account day-to-day use	Root only for account recovery — enforce via SCP as belt-and-suspenders

IAM Access Analyzer — what it finds and what it misses

Finding type	What Access Analyzer identifies	Limitation
External access — S3	Buckets accessible outside your account/org	Does not analyse policy permissions — only resource accessibility
External access — IAM role	Roles with cross-account or public trust	Does not flag over-permission within account
External access — KMS	KMS keys with cross-account decrypt permissions	Requires finding review — not auto-remediated
Unused access	Roles/users with permissions unused in 90 days	Generates least-privilege policy recommendations
Policy validation	Syntax errors and logic issues in policies	Does not catch semantic escalation paths (use Cloudsplaining)

NOTE

Access Analyzer is a first-pass tool. For escalation path analysis, also run Cloudsplaining or use the AWS IAM policy simulator. Access Analyzer finds WHO can access WHAT — not what they can DO once they have access.

SECTION 14

IAM Review Checklist & Scoring Matrix

Use this checklist for a structured IAM review. Score each control: Pass (P), Fail (F), or Not Applicable (N/A). Prioritise failures by severity.

ACCOUNT BASELINE		CRITICAL
	Root account has no active access keys	P / F / N/A
	Root account has hardware MFA enrolled	P / F / N/A
	Root account has no recent console logins (check CloudTrail)	P / F / N/A
	All privileged IAM users have MFA enforced	P / F / N/A
	No IAM users with console access + access keys simultaneously	P / F / N/A
CREDENTIAL HYGIENE		**HIGH**
	No access keys older than 90 days	P / F / N/A
	No unused access keys (last-used > 90 days)	P / F / N/A
	No secrets in Lambda environment variables	P / F / N/A
	No credentials in EC2 user-data scripts	P / F / N/A
	No long-lived keys for human users (SSO enforced)	P / F / N/A
POLICY & PERMISSIONS		**CRITICAL**
	No policies with Action:* Resource:* Effect:Allow	P / F / N/A
	iam:PassRole scoped to specific role ARNs (not Resource:*)	P / F / N/A
	iam:CreatePolicyVersion restricted to IAM governance role only	P / F / N/A
	iam:AttachUserPolicy/RolePolicy restricted to IAM governance role	P / F / N/A
	No inline policies on users or roles	P / F / N/A
	Permission boundaries attached to all developer/CI roles	P / F / N/A
	iam:* not granted to any non-IAM-admin identity	P / F / N/A
TRUST & FEDERATION		**HIGH**
	All cross-account trust policies require ExternalId	P / F / N/A
	All OIDC roles have branch + repo condition on token:sub	P / F / N/A
	No role trust policies with Principal:* without PrincipalOrgID condition	P / F / N/A
	Stale cross-account trust ARNs reviewed (no orphaned account refs)	P / F / N/A
	Service principals scoped to specific service (not Service:*)	P / F / N/A

ORG-LEVEL CONTROLS		HIGH
	SCP baseline deployed to all member OUs	P / F / N/A
	CloudTrail enabled in all regions, all accounts	P / F / N/A
	GuardDuty enabled in all regions, all accounts	P / F / N/A
	IAM Access Analyzer enabled, all findings reviewed	P / F / N/A
	Security Hub enabled with CIS AWS Benchmark standard	P / F / N/A
	AWS Config enabled with IAM-related managed rules	P / F / N/A

RESOURCE POLICIES		HIGH
	All S3 buckets have Block Public Access enabled	P / F / N/A
	No S3 bucket policies with Principal:* without org condition	P / F / N/A
	No KMS key policies with Principal:* without condition	P / F / N/A
	All Lambda resource policies reviewed for external access	P / F / N/A
	SQS/SNS topic policies reviewed for external access	P / F / N/A

Good luck on Friday. Think blast radius, think attacker perspective, think earliest break in the chain.